

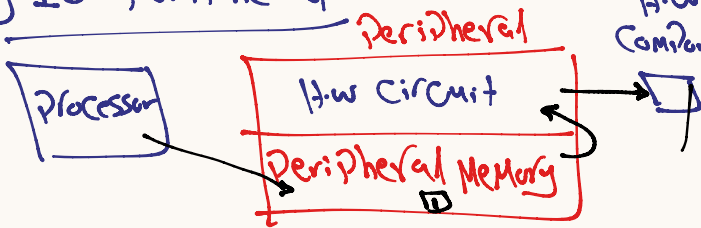
* Session 2 → ESC

1) Processor ✓

2) Memory ✓

→ RAM → SRAM → Data/Runtime
 → ROM → Flash → Code
 → EEPROM → Data/Code

3) Io Peripheral



* it's channel between Processor & H.W. Component

* any Peripheral must be inside Mc

* ex:

① DIO → Digital Input output

→ H.W. → Led / switch / buzzer
 7-seg / keypad / LCD
 Direction Motor / Digital sensor

② ADC → Analog to Digital.

→ H.W. → Analog sensor

③ DAC ④ UART ⑤ PWM
 ⑥ timer ⑦ SPI ⑧ I2C

Processor Communicate with memories?

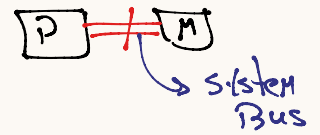
→ Flash → SRAM → EEPROM
 → IO Memory

* Buses

→ ① wire → SoB → White Board
 → ② track → SoB → PCB
 → ③ tunnel → SoC → chip (Mc)

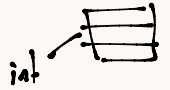
* System Bus

→ ① Data Bus
 → ② Address Bus
 → ③ Control Bus



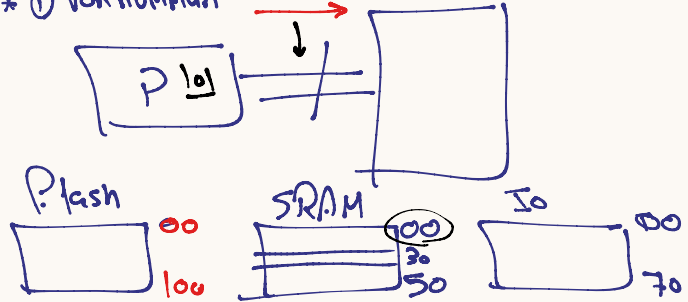
ex: → 32 bit → System Bus → 32 wire
 → int → 4 byte
 → Pointer → Fixed size

char * ptr



* System Bus Arch:

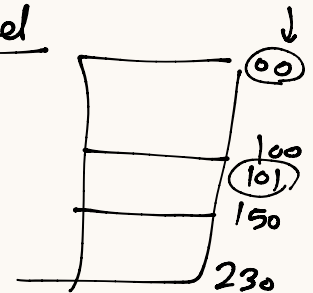
* ① Von numman



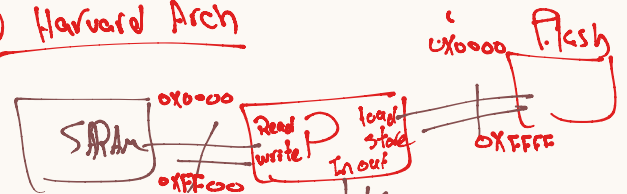
* Memory Mapped

* Low Cost

* time Accessing



② Harvard Arch



* ptr → 0x05 → I/O → 0x0000
 → 0x01FF

① Complex H.W ② High Cost

③ You Can Access any memory

if you write C
 Code to Access
 Address will be
 go to SRAM

if you need Access
 specific Address
 in another memory
 will be write Assembly

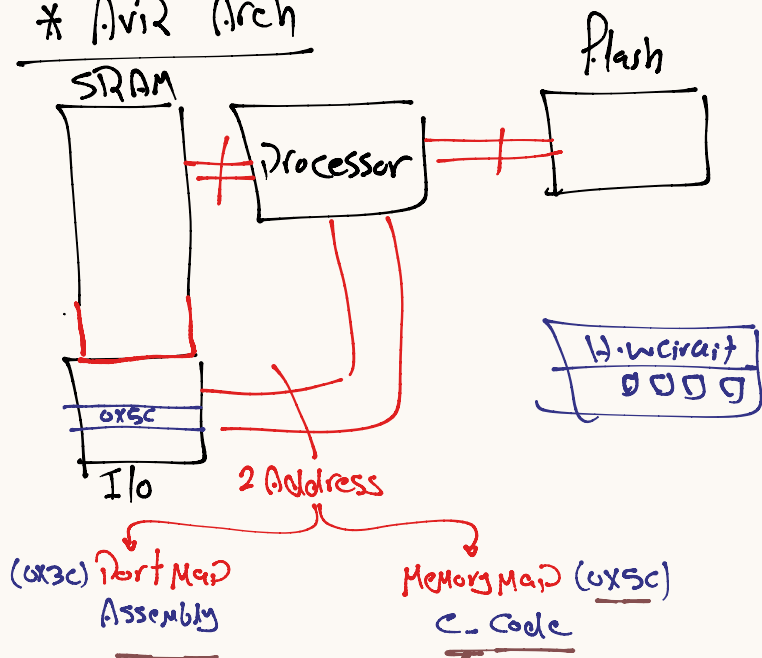
* Assembly

* Port mapped

* C Code

* Memory Mapped

* AVR Arch



* Clock Source

↳ must for Processor & Peripherals

ADC
timer
uart
SPI
I2C

* Internal → MC oscillator

↳ RC circuit → generate Pulse (clk)

* external → crystal oscillator

8MHz 16MHz

* You need know how switch between

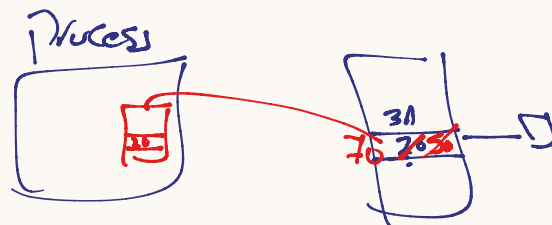
Clock Source

MC →
Fcu →
Development kit →

uint8_t *ptr; 0x3A;

uint8_t *ptr = (uint8_t *) 0x3A;

70 → 400 Address



volatile uint8_t *ptr: (volatile uint8_t *) 0x3A;

* ptr = 30;

#define DDR* ((volatile u8 *) 0x3A)

* ((volatile u8 *) 0x3A) = 50;

DDR = 70;

Processor
AVR
ARM
PIC

MC
Atmega32
stm877A
16F877A

